

Sistemas Operativos

Signals

1

1

Sinais – Interrupções – Eventos

- Um sinal é uma interrupção de software e notifica um processo que um evento ocorreu
- HW também pode criar eventos
 - Detetados por HW, recebidos no kernel e enviados para os processos como sinais;

- Exemplo:

- Teclas Ctrl+C gera um SIGINT



2

2

Tratamento de sinais

- Quando é recebido um sinal, ocorre 1 dos seguintes tratamento por omissão:
 - O sinal é ignorado
 - O processo é terminado
 - O processo é parado/suspenso
 - É gerado um ficheiro coredump (também para fazer debug) e o processo é terminado

3

3

Tratamento de sinais

- A função **signal** permite mudar o tratamento de um sinal
 - Definir outro tratamento pré-definido
 - Associar uma rotina do programa para tratar o signal
 - Ignorar o sinal (como se não tivesse existido)
- Alguns sinais não podem ser ignorados ou tratados
 - SIGKILL – Termina sempre um processo
 - SIGSTOP – Suspende um processo (!= SIGTSTP)
- Quando um processo inicia, os sinais estão no estado definido por omissão.

4

4

Alguns signals

Signal	Causa
SIGALRM	O relógio expirou
SIGFPE	Divisão por zero
SIGINT	O utilizador interrompe o processo (normalmente ctrl c)
SIGTSTP	O utilizador pára o processo – Terminal Stop – (normalmente ctrl Z)
SIGQUIT	O utilizador quer terminar o processo e provocar um core dump
SIGKILL	Signal para terminar o processo. Não pode ser tratado
SIGPIPE	O processo escreveu para um pipe que não tem recetores
SIGSEGV	Acesso a uma posição de memória inválida
SIGTERM	O utilizador pretende terminar o processo de forma ' <u>graciosa</u> '
SIGUSR1	Definido pelo utilizador
SIGUSR2	Definido pelo utilizador

5

5

Outras Funções

```
kill (pid, sig);
```

- Envia um determinado sinal (sig) a determinado processo (pid) – nome enganador?

```
alarm (segundos);
```

- Envia um sinal SIGALRM para o processo, depois de decorrerem o número de segundos especificados. Se o argumento for zero, o envio é cancelado.

```
pause();
```

- Aguarda a chegada de um sinal e espera que o tratamento termine.

6

6

Função Signal

```
sighandler_t signal(int signo, sighandler_t handler);
```

- `signo` identificador do signal para o qual se pretende definir um handler
- `handler` pode ser uma função ou macro: `SIG_DFL` (default action for signal in `signo`) ou `SIG_IGN` (ignore singal in `signo`)
- E.g.
 - `signal(SIGINT, nomeDaFuncao);` //SIGINT = Ctrl C
 - `signal(SIGINT, SIG_IGN);`
 - ...

7

7

Exemplo – SIGINT (ctrl c)

```
void sigint(int signum) {  
    char option;  
    printf("\n ^C pressed. Do you want to abort? y/n");  
  
    scanf(" %c", &option);  
  
    if (option == 'y') {  
        printf("Ok, bye bye!\n");  
        exit(0);  
    }  
}  
  
int main()  
{  
    // Redirects SIGINT to sigint()  
    signal(SIGINT, sigint);  
  
    // Do some work!  
    while (1) {  
        printf("Doing some work...\n");  
        sleep(1);  
    }  
    return 0;  
}
```

!NOTA!
Faltam os #includes...
Incluindo agora o
<signal.h>

```
Doing some work...  
Doing some work...  
Doing some work...  
^C  
^C pressed. Do you want to abort? y/ny  
Ok, bye bye!
```

8

8

Exemplo – SIGUSR1-2

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <signal.h>

void catcher(int signum){
    printf("\n\tCaught signal %d \n\n", signum);
}

int main(){
    pid_t pid;
    pid = fork();

    if (pid == 0){ //child
        //if SIGUSR1-2 signal is received, launch catcher
        signal(SIGUSR1, catcher);
        signal(SIGUSR2, catcher);

        printf("\nChild: waiting for signals...\n\n");
        pause(); //waits for a signal to arrive
        pause(); //waits for a signal to arrive
    }

    else{
        sleep(2);
        printf("\nParent: Going to send signal...\n");
        kill(pid, SIGUSR1); //sends signal SIGUSR1 to pid (child)
        sleep(2);
        printf("\nParent: Going to send signal...\n");
        kill(pid, SIGUSR2); //sends signal SIGUSR2 to pid (child)
        wait(NULL);
    }

    return 0;
}
```



```
Child: waiting for signals...
Parent: Going to send signal...
Caught signal 10
Parent: Going to send signal...
Caught signal 12
```

9

Função Signal

Nestes exemplos simples, ‘carregámos’ o handler com instruções... mas não é boa prática...

- Boas práticas dentro dos handlers:
 - O handler deve fazer o mínimo possível e não deve interagir com o user
 - Apenas sinaliza que um evento ocorreu e a main() trata da decisão
 - Definir flags globais
 - Usar tipos seguros (sig_atomic_t)
 - Usar funções async-signal-safe
 - Evitar: printf; scanf; exit... pode ter comportamento indefinido/deadlocks
 - Usar: write, read, _exit, getpid, kill ... (são async-signal-safe)

10

Função Signal

```
#include <stdio.h> <stdlib.h> <signal.h> <unistd.h>
volatile sig_atomic_t isSigintFlagged = 0;

void sigintHandler(int sig) {
    isSigintFlagged = 1;
}

int main(void) {
    signal(SIGINT, sigintHandler);

    while (1) {
        printf("A fazer trabalho...\n");
        usleep(100000); //0.1s

        if (isSigintFlagged) {
            isSigintFlagged = 0; //repôr valor

            char op;
            printf("\nTem a certeza que pretende sair? (s/n): ");
            scanf(" %c", &op);

            if (op == 's' || op == 'S') {
                printf("A sair...\n");
                exit(0);
            }

            printf("A Continuar a execução...\n");
        }
    }
}
```

O handler apenas assinala a flag...

...e a main faz a verificação da flag em pontos que considera seguros.
(podemos ter verificações em vários pontos da main)

11

11

Exercício Signals

Pretende-se desenvolver um jogo que testa a capacidade de o utilizador contar segundos mentalmente e tem o seguinte comportamento:

- Escreve no ecrã o número aleatório de segundos que devem ser contados (e.g., 5 a 10)
- Escreve '3,2,1,0' no ecrã, um número a cada 2 segundos: 3... 2... 1... 0 começa a contagem!
- O utilizador pressiona CTRL-Z quando achar que já passaram os segundos indicados
- O jogo deve escrever no ecrã uma mensagem a avisar se o utilizador acertou, ou não, na contagem do tempo. Após esta mensagem, o programa deve permitir iniciar outro jogo ao carregar na tecla ENTER
- Entre cada jogo, o CTRL-Z só deve ser válido enquanto é suposto contar mentalmente
- O utilizador pode pressionar CTRL-C a qualquer momento, para terminar o programa
- Pretende-se obter um output semelhante ao apresentado no próximo slide
- Pode recorrer a ferramentas de IA

13

13

Exercício Signals

```
Press CTRL-Z in 6 seconds!  
Countdown starting in 3 seconds... Get Ready...  
3... 2... 1...  
Start counting the seconds!!  
^Z  
  
=> 7 seconds elapsed! You were too slow...  
  
Press ENTER to continue!  
  
Press CTRL-Z in 8 seconds!  
Countdown starting in 3 seconds... Get Ready...  
Start counting the seconds!!  
^C  
  
^C pressed, do you want to abort? (y=yes)y  
Ok, bye bye!
```

14